Compiler Construction Lab - Assignment 14

Name: Dev Vachhani
PRN.: 24070122517
Batch: B3

Objective

To implement backpatching for Boolean expressions and generate intermediate code using jump instructions and labels.

Theory

Backpatching is a technique used in compiler design to generate intermediate code when the target of a jump instruction is not yet known. Instead of deciding every jump target immediately, the compiler temporarily leaves it incomplete and fills in the correct target later.

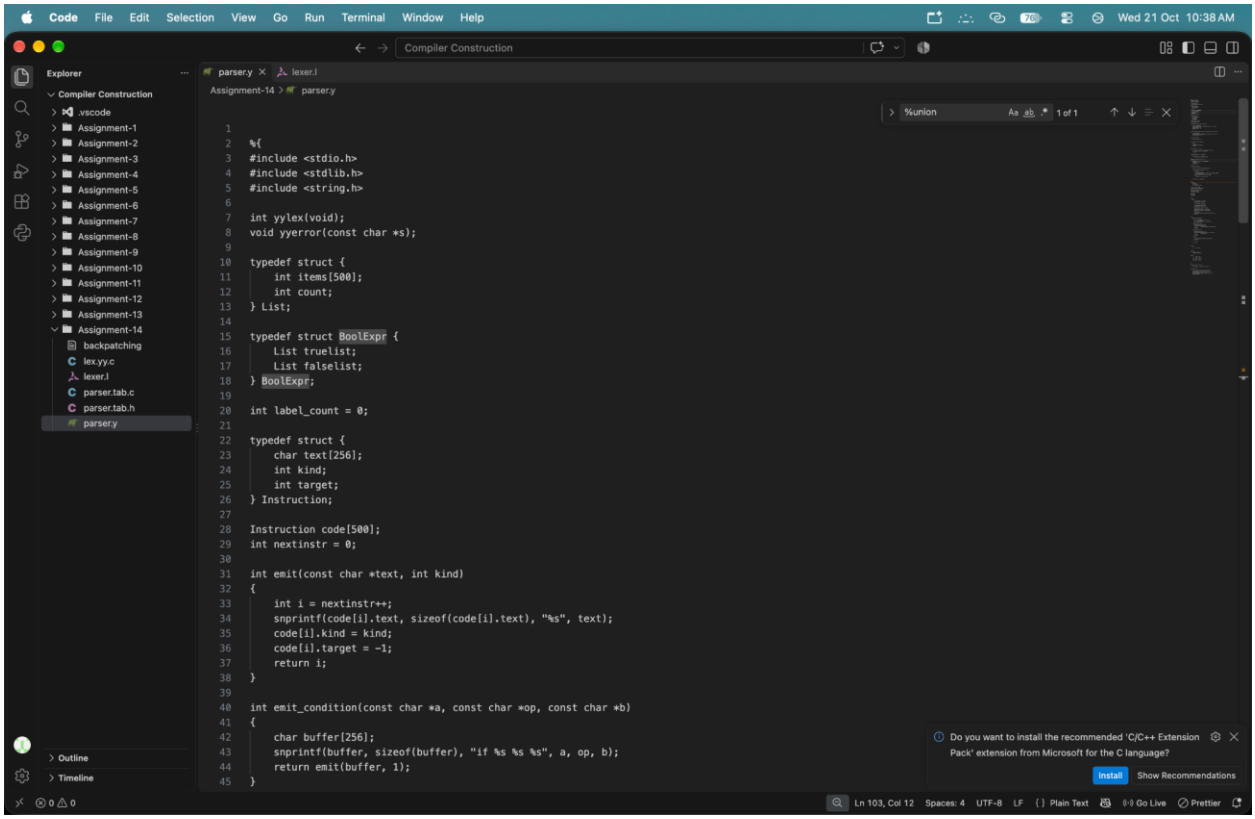
Backpatching is commonly used when generating intermediate code for Boolean expressions and control-flow statements. Each Boolean expression maintains two lists:

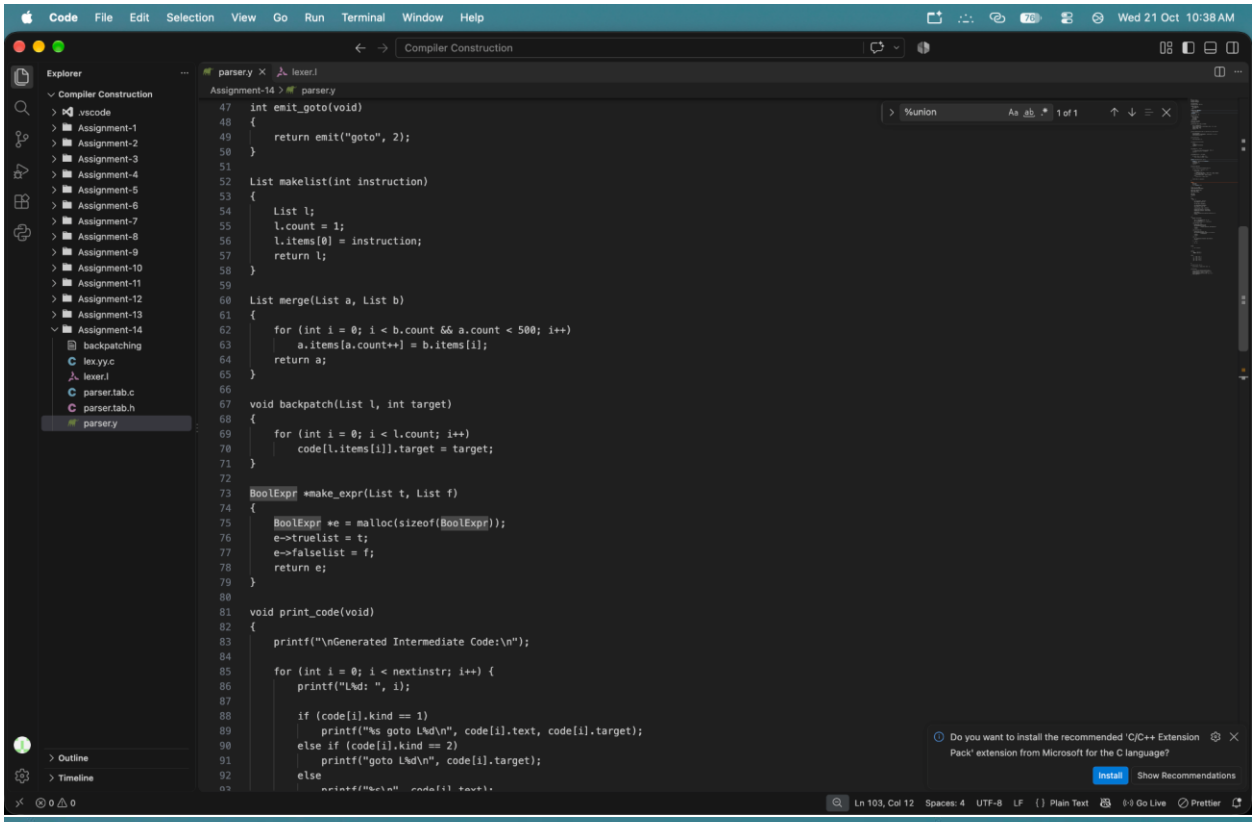
- True list: Contains jump instructions that must be patched with the address to follow when the expression evaluates to true.
- False list: Contains jump instructions that must be patched with the address to follow when the expression evaluates to false.

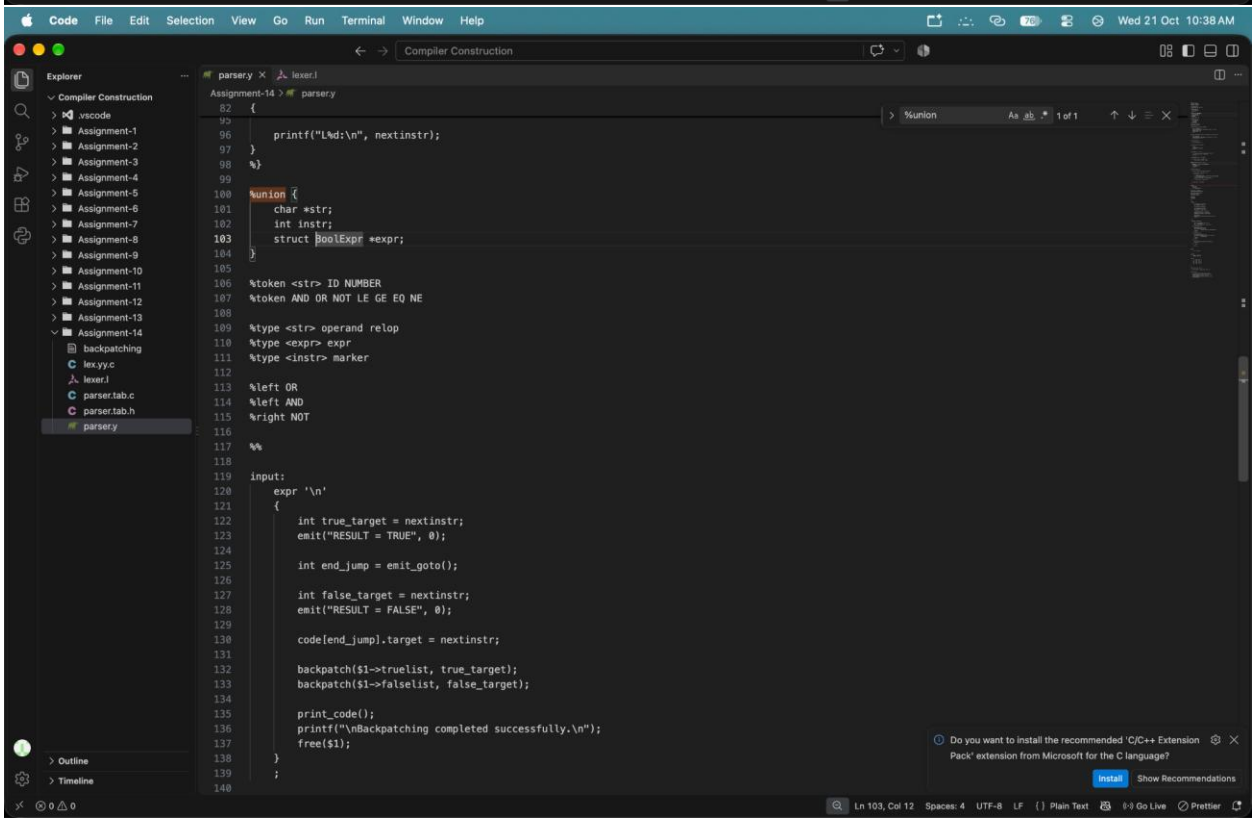
The && operator uses short-circuit evaluation: the second expression is evaluated only if the first expression is true. The || operator evaluates the second expression only if the first expression is false. The ! operator reverses the true and false outcomes.

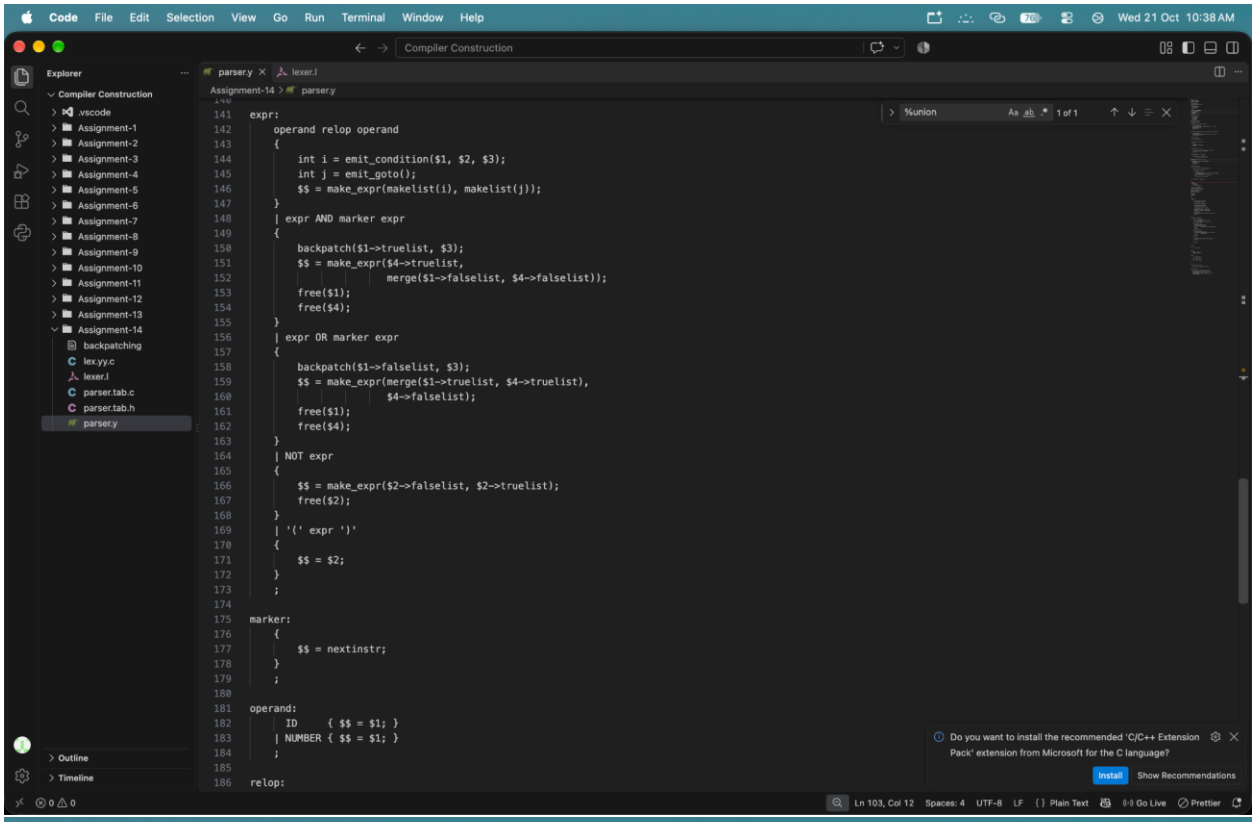
For example, for the expression a && b, the compiler first evaluates a. If a is false, execution jumps directly to the false destination. If a is true, execution continues to evaluate b. Backpatching fills in the jump destinations once the corresponding code locations are known.

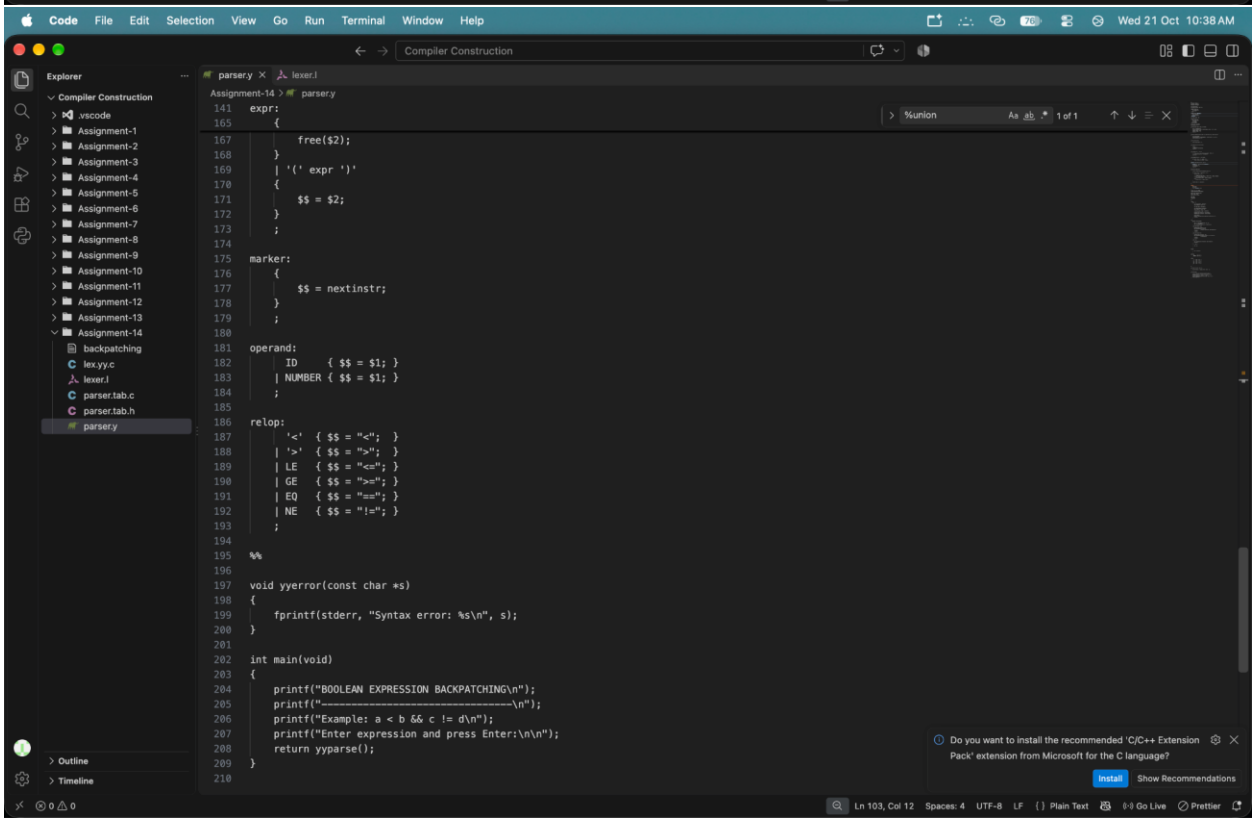
Screenshots and Execution Results

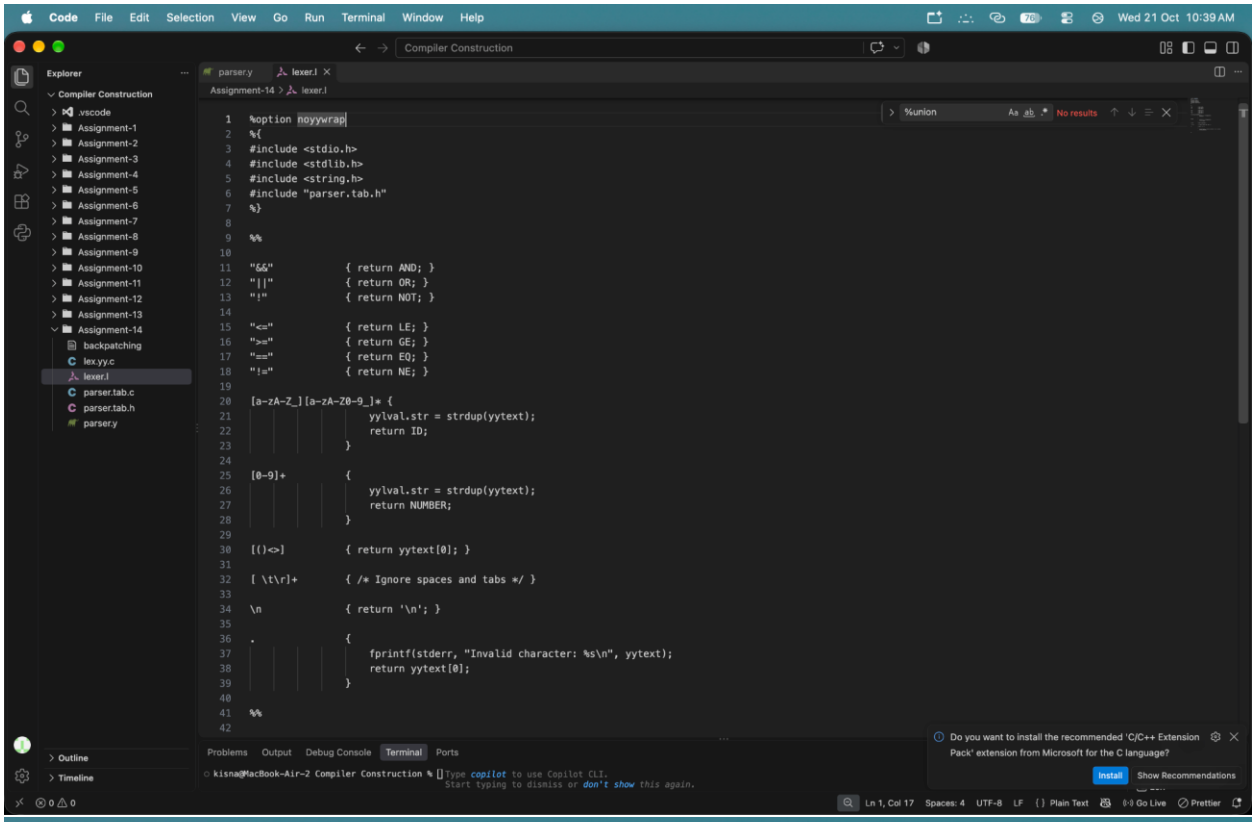


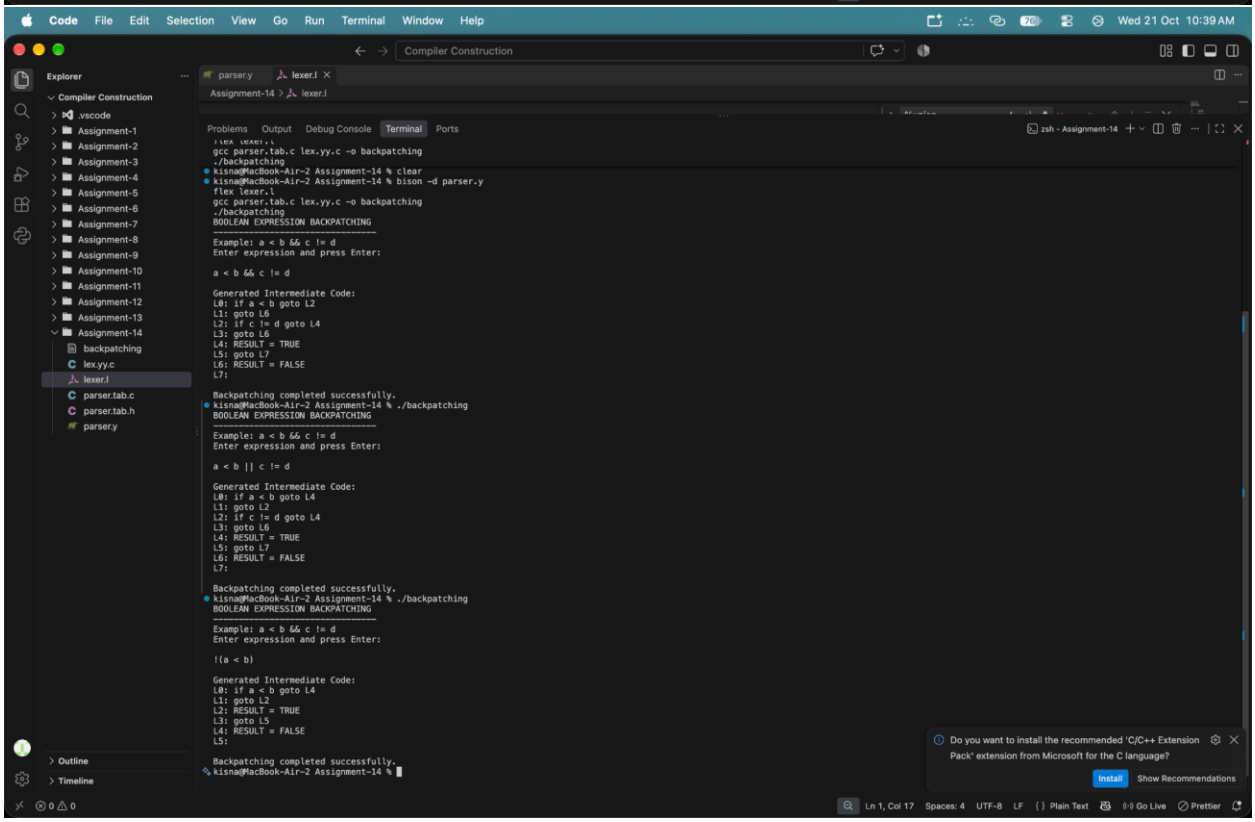

Code File Edit Selection View Go Run Terminal Window Help
Wed 21 Oct 10:38 AM
Compiler Construction
Explorer
Compiler Construction
parser.y
lexer.l
Assignment-14 > parser.y
%union
1 of 1
int emit_goto(void)
{
return emit("goto", 2);
}
List makelist(int instruction)
{
List l;
l.count = 1;
l.items[0] = instruction;
return l;
}
List merge(List a, List b)
{
for (int i = 0; i < b.count && a.count < 500; i++)
a.items[a.count++] = b.items[i];
return a;
}
void backpatch(List l, int target)
{
for (int i = 0; i < l.count; i++)
code[l.items[i]].target = target;
}
BoolExpr *make_expr(List t, List f)
{
BoolExpr *e = malloc(sizeof(BoolExpr));
e->truelist = t;
e->falselist = f;
return e;
}
void print_code(void)
{
printf("\nGenerated Intermediate Code:\n");
for (int i = 0; i < nextinstr; i++) {
printf("L%d: ", i);
if (code[i].kind == 1)
printf("%s goto L%d\n", code[i].text, code[i].target);
else if (code[i].kind == 2)
printf("goto L%d\n", code[i].target);
else
Do you want to install the recommended 'C/C++ Extension Pack' extension from Microsoft for the C language?
Install
Show Recommendations
Ln 103, Col 12
Spaces: 4
UTF-8
LF
Plain Text
Go Live
Prettier


Code File Edit Selection View Go Run Terminal Window Help
Wed 21 Oct 10:38 AM
Compiler Construction
Explorer
Compiler Construction
parser.y
lexer.l
Assignment-14 > parser.y
%union
1 of 1
{
printf("L%d:\n", nextinstr);
}
%}
%union {
char *str;
int instr;
struct BoolExpr *expr;
}
%token <str> ID NUMBER
%token AND OR NOT LE GE EQ NE
%type <str> operand relop
%type <expr> expr
%type <instr> marker
%left OR
%left AND
%right NOT
%%
input:
expr '\n'
{
int true_target = nextinstr;
emit("RESULT = TRUE", 0);
int end_jump = emit_goto();
int false_target = nextinstr;
emit("RESULT = FALSE", 0);
code[end_jump].target = nextinstr;
backpatch($1->truelist, true_target);
backpatch($1->falselist, false_target);
print_code();
printf("\nBackpatching completed successfully.\n");
free($1);
}
;
Do you want to install the recommended 'C/C++ Extension Pack' extension from Microsoft for the C language?
Install
Show Recommendations
Ln 103, Col 12
Spaces: 4
UTF-8
LF
Plain Text
Go Live
Prettier


Code File Edit Selection View Go Run Terminal Window Help
Wed 21 Oct 10:38 AM
Compiler Construction
Explorer
Compiler Construction
.vscode
Assignment-1
Assignment-2
Assignment-3
Assignment-4
Assignment-5
Assignment-6
Assignment-7
Assignment-8
Assignment-9
Assignment-10
Assignment-11
Assignment-12
Assignment-13
Assignment-14
backpatching
lex.yy.c
lexer.l
parser.tab.c
parser.tab.h
parser.y
parser.y
lexer.l
Assignment-14 > parser.y
%union
1 of 1
expr:
operand relop operand
{
int i = emit_condition($1, $2, $3);
int j = emit_goto();
$$ = make_expr(makelist(i), makelist(j));
}
| expr AND marker expr
{
backpatch($1->truelist, $3);
$$ = make_expr($4->truelist,
merge($1->falselist, $4->falselist));
free($1);
free($4);
}
| expr OR marker expr
{
backpatch($1->falselist, $3);
$$ = make_expr(merge($1->truelist, $4->truelist),
$4->falselist);
free($1);
free($4);
}
| NOT expr
{
$$ = make_expr($2->falselist, $2->truelist);
free($2);
}
| '(' expr ')'
{
$$ = $2;
}
;
marker:
{
$$ = nextinstr;
}
;
operand:
ID { $$ = $1; }
| NUMBER { $$ = $1; }
;
relop:
Do you want to install the recommended 'C/C++ Extension Pack' extension from Microsoft for the C language?
Install
Show Recommendations
Outline
Timeline
Ln 103, Col 12
Spaces: 4
UTF-8
LF
Plain Text
Go Live
Prettier


Code File Edit Selection View Go Run Terminal Window Help
Wed 21 Oct 10:38 AM
Compiler Construction
Explorer
Compiler Construction
.vscode
Assignment-1
Assignment-2
Assignment-3
Assignment-4
Assignment-5
Assignment-6
Assignment-7
Assignment-8
Assignment-9
Assignment-10
Assignment-11
Assignment-12
Assignment-13
Assignment-14
backpatching
lex.yy.c
lexer.l
parser.tab.c
parser.tab.h
parser.y
parser.y
lexer.l
Assignment-14 > parser.y
%union
1 of 1
expr:
{
free($2);
}
| '(' expr ')'
{
$$ = $2;
}
;
marker:
{
$$ = nextinstr;
}
;
operand:
ID { $$ = $1; }
| NUMBER { $$ = $1; }
;
relop:
'<' { $$ = "<"; }
| '>' { $$ = ">"; }
| LE { $$ = "<="; }
| GE { $$ = ">="; }
| EQ { $$ = "=="; }
| NE { $$ = "!="; }
;
%%
void yyerror(const char *s)
{
fprintf(stderr, "Syntax error: %s\n", s);
}
int main(void)
{
printf("BOOLEAN EXPRESSION BACKPATCHING\n");
printf("--------------------------------\n");
printf("Example: a < b && c != d\n");
printf("Enter expression and press Enter:\n\n");
return yyparse();
}
Do you want to install the recommended 'C/C++ Extension Pack' extension from Microsoft for the C language?
Install
Show Recommendations
Outline
Timeline
Ln 103, Col 12
Spaces: 4
UTF-8
LF
Plain Text
Go Live
Prettier


Code File Edit Selection View Go Run Terminal Window Help
Wed 21 Oct 10:39 AM
Compiler Construction
Explorer
Compiler Construction
.vscode
Assignment-1
Assignment-2
Assignment-3
Assignment-4
Assignment-5
Assignment-6
Assignment-7
Assignment-8
Assignment-9
Assignment-10
Assignment-11
Assignment-12
Assignment-13
Assignment-14
backpatching
lex.yy.c
lexer.l
parser.tab.c
parser.tab.h
parser.y
parser.y
lexer.l
Assignment-14 > lexer.l
%option noyywrap
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "parser.tab.h"
%}
%%
"&&" { return AND; }
"||" { return OR; }
"!" { return NOT; }
"<=" { return LE; }
">=" { return GE; }
"==" { return EQ; }
"!=" { return NE; }
[a-zA-Z_][a-zA-Z0-9_]* {
yylval.str = strdup(yytext);
return ID;
}
[0-9]+ {
yylval.str = strdup(yytext);
return NUMBER;
}
[()<>] { return yytext[0]; }
[\t\r]+ { /* Ignore spaces and tabs */ }
\n { return '\n'; }
. {
fprintf(stderr, "Invalid character: %s\n", yytext);
return yytext[0];
}
%%
%union
No results
Problems Output Debug Console Terminal Ports
kisna@MacBook-Air~2 Compiler Construction %
Type copilot to use Copilot CLI.
Start typing to dismiss or don't show this again.
Do you want to install the recommended 'C/C++ Extension Pack' extension from Microsoft for the C language?
Install
Show Recommendations
Outline
Timeline
Ln 1, Col 17 Spaces: 4 UTF-8 LF Plain Text Go Live Prettier


Code File Edit Selection View Go Run Terminal Window Help
Wed 21 Oct 10:39 AM
Compiler Construction
Explorer
Compiler Construction
.vscode
Assignment-1
Assignment-2
Assignment-3
Assignment-4
Assignment-5
Assignment-6
Assignment-7
Assignment-8
Assignment-9
Assignment-10
Assignment-11
Assignment-12
Assignment-13
Assignment-14
backpatching
lex.yy.c
lexer.l
parser.tab.c
parser.tab.h
parser.y
parser.y
lexer.l
Assignment-14 > lexer.l
Problems Output Debug Console Terminal Ports
zsh - Assignment-14
gcc parser.tab.c lex.yy.c -o backpatching
./backpatching
kisna@MacBook-Air~2 Assignment-14 % clear
kisna@MacBook-Air~2 Assignment-14 % bison -d parser.y
flex lexer.l
gcc parser.tab.c lex.yy.c -o backpatching
./backpatching
BOOLEAN EXPRESSION BACKPATCHING
Example: a < b && c != d
Enter expression and press Enter:
a < b && c != d
Generated Intermediate Code:
L0: if a < b goto L2
L1: goto L6
L2: if c != d goto L4
L3: goto L6
L4: RESULT = TRUE
L5: goto L7
L6: RESULT = FALSE
L7:
Backpatching completed successfully.
kisna@MacBook-Air~2 Assignment-14 % ./backpatching
BOOLEAN EXPRESSION BACKPATCHING
Example: a < b && c != d
Enter expression and press Enter:
a < b || c != d
Generated Intermediate Code:
L0: if a < b goto L4
L1: goto L2
L2: if c != d goto L4
L3: goto L6
L4: RESULT = TRUE
L5: goto L7
L6: RESULT = FALSE
L7:
Backpatching completed successfully.
kisna@MacBook-Air~2 Assignment-14 % ./backpatching
BOOLEAN EXPRESSION BACKPATCHING
Example: a < b && c != d
Enter expression and press Enter:
!(a < b)
Generated Intermediate Code:
L0: if a < b goto L4
L1: goto L2
L2: RESULT = TRUE
L3: goto L5
L4: RESULT = FALSE
L5:
Backpatching completed successfully.
kisna@MacBook-Air~2 Assignment-14 %
Do you want to install the recommended 'C/C++ Extension Pack' extension from Microsoft for the C language?
Install
Show Recommendations
Outline
Timeline
Ln 1, Col 17 Spaces: 4 UTF-8 LF Plain Text Go Live Prettier